



MODULE 6

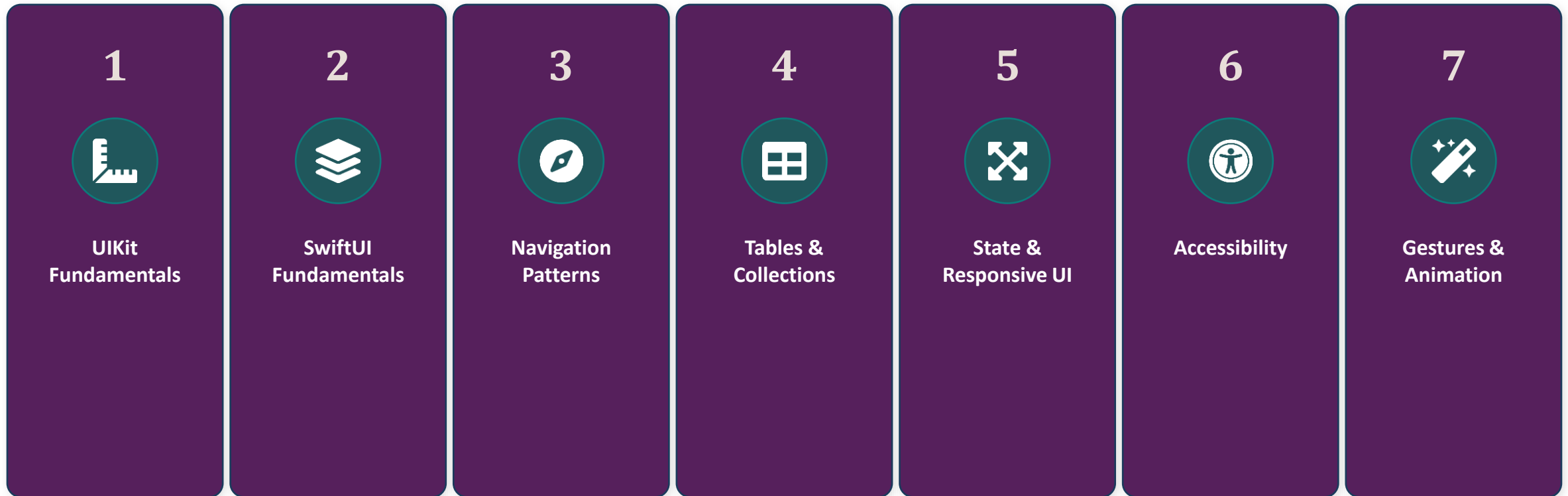
Building iOS User Interfaces

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 02 | iOS Development with Swift

Module 6 Roadmap

Seven blocks. By the end: a working, accessible, animated PNC Mobile screen.



UIKit Fundamentals

- Views & the Responder Chain
- Auto Layout
- Stack Views



UIView, View Controllers & the Lifecycle



UIView

The base class for anything drawn on screen - buttons, labels, containers all inherit from it.



UIViewController

Owns one screen's worth of views and manages their lifecycle.



viewDidLoad()

Called once, after the view hierarchy is loaded into memory. Do one-time setup here.



viewWillAppear()

Called every time the screen is about to become visible - refresh data here.



UIView Lifecycle Events

A UIView goes through many events throughout its lifecycle. The runtime will invoke these events, if you define them.

loadView()

viewDidLoad
(once)

viewWillAppear
(every time)

viewDidAppear
(every time)

viewWillDisappear
(every time)

viewDidDisappear
(every time)



Auto Layout: The Constraint That Gets Forgotten

- This code results in constraints being silently ignored
- It also has Auto Layout warnings in the console

Code Bite

```
let label = UILabel()
label.text = "Balance"
view.addSubview(label)

NSLayoutConstraint.activate([
    label.topAnchor.constraint(
        equalTo: view.safeAreaLayoutGuide.topAnchor,
        constant: 20),
    label.leadingAnchor.constraint(
        equalTo: view.leadingAnchor, constant: 16)
])
```



translateAutoResizingMaskIntoConstraints

- This is a Bool property every UIView has
- It determines whether a view's old-style autoresizing mask is automatically converted into modern Auto Layout constraints
- When true (Default for code), the system automatically generates immutable constraints (of type NSLayoutConstraint) to match the view's current frame and autoresizingMask
 - If you try to add your own programmatic layout constraints on top of this, they will almost always conflict, causing layout crashes or unexpected UI breakages
- When false: The autoresizing mask is ignored.
 - The system stops generating automatic constraints, leaving you with full, explicit control to define the view's size and position using Auto Layout anchors or constraints



Rule of Thumb

Context	Default Value	What you should do
Programmatic views let v = UIView()	true	Set it to false right before adding your own constraints
Interface Builder (Storyboards and XIBs)	false	Leave it alone (the system flips it to false the moment you add your own constraints in the designer)
System-managed views (UITableViewCell, etc.)	true	Leave it alone (changing it on views managed directly by UIKit can break their built-in layout engines)



Auto Layout: The Fix

- The fix is to disable translation of the Autoresizing Mask into Constraints

Code Bite

```
let label = UILabel()
label.text = "Balance"
label.translatesAutoresizingMaskIntoConstraints
= false
view.addSubview(label)

NSLayoutConstraint.activate([
    label.topAnchor.constraint(
        equalTo: view.safeAreaLayoutGuide.topAnchor,
        constant: 20),
    label.leadingAnchor.constraint(
        equalTo: view.leadingAnchor, constant: 16)
])
```



UIStackView

- Most real screens are just rows and columns of things
- UIStackView handles that layout without needing constraints between every sibling item
- NSLayoutConstraint are only needed on the top-level UIStackView
 - It will automatically lay out its child views
- You can nest them inside of each other
 - E.g. a vertically oriented stack view containing horizontally oriented stack views to create both rows and columns



UIStackView: Auto Layout Without the Boilerplate

```
let stack = UIStackView(arrangedSubviews: [  
    balanceLabel, accountNumberLabel, statusIcon  
])  
stack.axis = .vertical  
stack.spacing = 12  
stack.alignment = .leading  
stack.translatesAutoresizingMaskIntoConstraints = false  
view.addSubview(stack)
```

```
NSLayoutConstraint.activate([  
    stack.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor, constant: 20),  
    stack.leadingAnchor.constraint(equalTo: view.leadingAnchor, constant: 16),  
    stack.trailingAnchor.constraint(equalTo: view.trailingAnchor, constant: -16)  
])
```



Block 1 - UIKit Fundamentals

Skill Check ✓

- What's the one-line fix for a programmatically created view whose constraints appear to do nothing?
- Which lifecycle method runs once, and which runs every time a screen becomes visible?
- When would you reach for `UIStackView` instead of hand-written `NSLayoutConstraints`?



SwiftUI Fundamentals

- The View Protocol
- @State & @Binding
- @StateObject, @ObservedObject & @EnvironmentObject



The SwiftUI View Protocol

- A SwiftUI View is not a persistent object the way a UIView is
 - It is a lightweight, disposable description of what the UI should look like right now, given the current state
- SwiftUI recreates these structs constantly as state changes
- That is expected and cheap, not a performance problem to work around
- The View protocol simply requires one property called body
 - It should return a description of what should be onscreen
- Everything you see on screen is a View
 - Text label? - View
 - Button? - View
 - Image? - View
 - Invisible containers for layout? - View



SwiftUI Code

```
struct BalanceCardView: View {  
    let accountName: String  
    let balance: Decimal  
  
    var body: some View {  
        VStack(alignment: .leading, spacing: 8) {  
            Text(accountName).font(.headline)  
            Text(balance, format: .currency(code: "USD")).font(.largeTitle.bold())  
        }  
        .padding()  
        .background(.thinMaterial)  
        .clipShape(RoundedRectangle(cornerRadius: 16))  
    }  
}
```



@State

- @State creates and owns a piece of data within a view
- The \$ prefix (as in \$amount) creates a Binding from a @State property
 - It's the mechanism that lets AmountField modify TransferAmountView's amount without owning it itself.
- When a @State value changes, SwiftUI automatically re-renders the owning view and any child views relying on that data

Code Bite

```
struct TransferAmountView: View {  
    @State private var amount = ""  
  
    var body: some View {  
        AmountField(amount: $amount)  
    }  
}
```



@Binding

- @Binding references and modifies data owned by another view
- The \$ prefix (as in \$amount) creates a Binding from a @Binding property
 - It's the mechanism that lets TextField modify AmountField's amount without owning it itself.
- When a child updates a @Binding value, it directly modifies the parent's @State, causing both views to sync and re-render instantly

Code Bite

```
struct AmountField: View {  
    @Binding var amount: String  
  
    var body: some View {  
        TextField("$0.00", text: $amount)  
    }  
}
```



@State vs @Binding

Feature	@State	@Binding
Ownership	Owns the data	References external data
Storage	Allocates physical memory for the value	Stores a reference pointer to the data
Initialization	Needs an initial default value	Cannot have a default value
Access Scope	Best marked as private to the current view	Passed in from a parent view
Syntax Usage	Read: variable Write link: \$variable	Read: variable Write link: \$variable



@StateObject vs. @ObservedObject vs. @EnvironmentObject

Wrapper	Use When
@StateObject	This view CREATES and owns the object. It survives view re-creation
@ObservedObject	This view was HANDED the object by a parent. Does not own its lifecycle
@EnvironmentObject	The object is injected from far up the view tree, shared implicitly

Block 2 - SwiftUI Fundamentals

Skill Check ✓

- A view creates a SessionManager itself. Which property wrapper does it use?
- A child view receives a SessionManager from its parent's initializer. Which wrapper does it use?
- What does the \$ prefix on a @State property actually produce?



Navigation Patterns

- UINavigationController
- NavigationStack & NavigationLink
- Tab Bars & Modals



UIKit: UINavigationController

- The window's rootViewController is usually setup in the SceneDelegate
- pushViewController() is used to navigate to a detail screen when you want to be able to return to the current screen
- popViewController() will navigate back to the screen displayed before the call to pushViewController()

Code Bite

```
let nav = UINavigationController(  
    rootViewController: AccountListViewController()  
)  
window?.rootViewController = nav  
  
let detailVC =  
    AccountDetailViewController(account: account)  
navigationController?.pushViewController(detailVC,  
    animated: true)  
  
navigationController?.popViewController(animated:  
    true)
```



SwiftUI: NavigationStack & NavigationLink

- Wrap the entire display in a NavigationStack
 - Create a List() out of your array of objects
- Wrap each element's display in a NavigationLink
 - Specify the object as its value property
 - Display content of your choice inside
- Set the navigationDestination on the List
 - Its for property lets you specify the data type being sent to the destination content
 - The closure lets you specify what content to display when an item is clicked

Code Bite

```
var body: some View {  
    NavigationStack {  
        List(accounts) { account in  
            NavigationLink(value: account) {  
                AccountRow(account: account)  
            }  
        }.navigationDestination(for: Account.self) {  
            account in  
                AccountDetailView(account: account)  
        }.navigationTitle("Accounts")  
    }  
}
```



Tab Bars & Modals

Pattern	UIKit	SwiftUI
Tab Bar	UITabBarController	TabView { . . . }
Modal / Sheet	present(_:animated:)	.sheet(isPresented:) { . . . }
Full-Screen Cover	modalPresentationStyle = .fullScreen	.fullScreenCover(isPresented:) { . . . }



Block 3 - Navigation Patterns

Skill Check ✓

- What SwiftUI modifier connects a NavigationLink's value to the view that should display it?
- Which UIKit method pushes a new screen onto the navigation stack?
- When would you reach for a full-screen cover instead of a sheet?



Table Views & Collection Views

- UITableView: DataSource & Delegate
- SwiftUI List
- UICollectionView & Diffable Data Sources



UITableView

- The UITableView is responsible for displaying a list of cells
- Each cell is created to display UI based on an element in a collection of data
- It efficiently recycles cells as they are scrolled offscreen
 - Offscreen cells are re-used as the cells that are being scrolled onscreen

Code Bite

```
let tableView = UITableView()

self.view.addSubview(tableView)
tableView.translatesAutoresizingMaskIntoConstraints = false

NSLayoutConstraint.activate([
    tableView.topAnchor.constraint(. . . ),
    . . .
])
```



UITableViewDataSource Protocol

- This protocol provides the data and rendering methods for the UITableView
 - It determines "what goes into each row"
- One method returns the total number of data points in the collection
- The other method configures a single cell to display the content you wish

Code Bite

```
class MyClass: UITableViewDataSource {  
    func tableView(_ tableView: UITableView,  
        numberOfRowsInSection section: Int) ->  
        Int {  
        return myDataColl.count  
    }  
    func tableView(_ tableView: UITableView,  
        cellForRowAt indexPath: IndexPath) ->  
        UITableViewCell {  
        // get the cell and set its content  
    }  
}
```



Configuring Content

- dequeueReusableCell will get an available cell to recycle or create a new one, as necessary
- The identifier used ("cell" in this case) is specified in the configuration of the UITableView using its register() method
- You can also create your own custom cell class for specialized content

Code Bite

```
// in the cellForRowAt method:  
let cell = tableView.dequeueReusableCell(  
    withIdentifier: "cell", for: indexPath)  
  
cell.textLabel?.text = "some content"  
  
return cell
```

